

C# Memory Boost Lecture

Namespaces & Keywords — Explained Simply

This lecture covers four things in a beginner-friendly way: what a namespace is and why it matters, what keywords are in C# and what they do, the base structure of a simple C# program explained line by line, and what the **using** directive actually means.

Goal: after reading this, you should be able to look at any basic C# file and explain what every single line is doing — not just recognize it, but actually understand it.

1. What is a Namespace?

A **namespace** is just a named container (a box) that groups related classes, methods, and other code together, so names don't clash with each other. That's it. It does not run any logic by itself — it only organizes code and prevents naming conflicts.

Real-world example

Think of your phone's contact list. You might have a friend named "Rahul" and a colleague also named "Rahul." To tell them apart, you save them as **College.Rahul** and **Office.Rahul**. "College" and "Office" work like namespaces here — same name, but no confusion, because the container tells you which one you mean.

In code, the same problem happens: two different libraries might both have a class called **Logger**. If both are in your project, the compiler wouldn't know which one you mean. Namespaces fix this by putting each **Logger** in its own labeled box, e.g. **CompanyA.Logger** vs **CompanyB.Logger**.

Why namespaces are needed — the actual job they do

- Avoid naming collisions between classes with the same name.
- Organize a large project into logical groups (e.g. all database code in one namespace, all UI code in another).
- Make code easier to find and reuse — you know exactly where a class "lives" by its namespace path.
- Control access — you only get to use a class directly once you either write its full path or import (using) its namespace.

Example

```
namespace SchoolSystem
{
    class Student
    {
        // student-related code goes here
    }
}
```

Here, **Student** is placed inside the **SchoolSystem** box. Its full, official name is really **SchoolSystem.Student**, not just **Student**. If another part of a bigger program has a different **Student** class inside a **HospitalSystem** namespace, there is zero conflict — they're two different rooms in the same building.

2. Base Structure of a C# Program — Line by Line

Below is the smallest realistic C# program. Every single line is broken down under it — treat this as the skeleton that almost every C# file is built on top of.

```
using System;

namespace SchoolSystem
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, Bumba!");
        }
    }
}
```

Line	What it means
using System;	Imports the built-in System namespace so you can use its tools (like Console) without typing the full path every time. Explained fully in Section 3.
namespace SchoolSystem	Declares a container/box named SchoolSystem . Everything inside the curly braces below belongs to this box.
{ ... }	Curly braces mark the start and end of a block of code. Every namespace, class, and method has its own opening { and matching closing }.
class Program	A class is a blueprint/template for an object. Here we define one called Program — by convention, this is the class that holds your app's entry point.
static void Main(string[] args)	This is the Main method — the exact spot where the program starts running. static means you don't need to create an object to run it. void means it returns nothing. string[] args lets you pass command-line arguments in, if needed.
Console.WriteLine(...)	Calls the WriteLine method that lives inside the Console class (which lives inside the System namespace) to print text to the screen, then moves to a new line.
;	A semicolon ends every statement in C#. Forget it and the code won't compile.

Execution order in plain words: the program starts at **Main**, runs each statement top to bottom inside it, and stops once it reaches the closing brace of **Main**.

3. What does `using` mean?

The **using** directive at the top of a file tells the compiler: "I want to use the classes inside this namespace, so let me refer to them by their short name instead of typing the full path every time."

Without `using`

```
System.Console.WriteLine("Hi");
```

With `using System;` at the top

```
using System;  
  
Console.WriteLine("Hi");
```

Both lines do the exact same thing. **using System;** just saves you from writing **System.** before every class that belongs to that namespace, throughout the whole file.

Real-world example

This is like saving a long international phone number as a contact named "Mom" on your phone. The real number (**System.Console**) still exists and still works — you've just created a short alias so you don't retype the whole thing every time.

Second meaning: `using(...)` blocks

There is a second, different job for the word **using**: a **using(...)** block automatically closes/releases a resource (like a file or database connection) as soon as you're done with it, even if an error happens. This is unrelated to namespace importing — same keyword, different purpose.

```
using (var reader = new StreamReader("data.txt"))  
{  
    string content = reader.ReadToEnd();  
}  
// file is automatically closed here, guaranteed
```

4. C# Keywords — What They Are and What They Do

A **keyword** is a reserved word that already has a fixed meaning built into the C# language. You cannot use a keyword as the name of your own variable, class, or method (e.g. you can't name a variable **class** or **int**) because the compiler has already claimed that word for itself. C# has around **100 keywords** total (roughly 79 reserved keywords plus about 25 contextual keywords that only mean something special in certain situations).

Data type keywords

Keyword	What it does
int	Stores whole numbers (e.g. 5, -12, 1000).
double / float	Stores decimal/fractional numbers.
string	Stores text.
bool	Stores true or false only.
char	Stores a single character, e.g. 'A'.
void	Means a method returns nothing.
var	Lets the compiler figure out the type automatically from the value.

Class / structure keywords

Keyword	What it does
class	Defines a blueprint for creating objects.
struct	Like a lightweight class, usually for small data groups.
interface	Defines a contract of methods a class must implement.
enum	Defines a fixed list of named constant values.
namespace	Groups related code together (Section 1).

Access modifier keywords

Keyword	What it does
public	Accessible from anywhere in the program.
private	Accessible only inside the same class.
protected	Accessible in the same class and its child classes.

<code>internal</code>	Accessible only within the same project/assembly.
<code>static</code>	Belongs to the class itself, not to any one object.

Control flow keywords

Keyword	What it does
<code>if / else</code>	Runs code only when a condition is true / otherwise.
<code>switch / case</code>	Chooses one path out of several based on a value.
<code>for / foreach</code>	Repeats code a set number of times, or once per item in a collection.
<code>while / do</code>	Repeats code as long as a condition stays true.
<code>break</code>	Immediately exits a loop or switch block.
<code>continue</code>	Skips to the next round of a loop.
<code>return</code>	Sends a value back and exits the current method.

Object / memory keywords

Keyword	What it does
<code>new</code>	Creates a new object/instance in memory.
<code>this</code>	Refers to the current object itself.
<code>base</code>	Refers to the parent class, used to call its members.
<code>null</code>	Represents "no value" / nothing assigned.
<code>try / catch / finally</code>	Handles errors gracefully instead of crashing.
<code>using</code>	Imports a namespace, or auto-releases a resource (Section 3).

You don't need to memorize all of these today — recognize the pattern instead: type keywords describe data, class-family keywords describe structure, access keywords describe visibility, and flow keywords describe the order code runs in.

Quick Recap

- **Namespace** = a labeled box that groups code and prevents naming clashes.
- **using [Namespace];** = import that box so you can use short names instead of full paths.
- **using (...) { }** = a different job — auto-closes a resource when the block ends.
- **Keyword** = a reserved word with a fixed meaning that you can't reuse as your own name.
- Base program flow: **using** imports → **namespace** groups → **class** defines a blueprint → **Main** is where execution starts → statements run top to bottom, each ending in ;.